

Infrastructure Module Reference

This section inventories every Layer-1 subdirectory returned by `28 discover_infrastructure_modules(repo_root)`. File totals use 769 Python sources across `infra` + 10,163 `infra` tests guarding them. Documentation Duality = paired `README.md` + `AGENTS.md`; optional `SKILL.md` manifests feed `python -m infrastructure.skills`.

Infrastructure Module Reference II

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
autoresearch	11			build_autoresearch_plan, readiness validation CLI
benchmark	4			Template harness scoring
config	0			+ comparative gates
core	133			Repository defaults + hardened templates
				get_logger,
				load_config,
docker	0			TemplateError
				Containerisation
doctor	14			scaffolding
				Checkout
documentation	23			diagnose/fix/undo repairs
				FigureManager, generate
fonds	6			_glossary
llm	55			—
				Ollama helpers,
				sanitization, review +
				translation pipelines
logrotate.d	0			Rotation snippets
				(documentation-first)
methods	8			build_methods_orchestration_plan,
				methods-stage contracts +
				validation
orchestration	12			PipelineRunner, entry
				point for ./run.sh
project	42			discover_projects,
				workspace management
prose	9			Markdown readability +
				prose tooling
provenance	8			—
publishing	84			Zenodo. executable

Alphabetical summaries

Alphabetical summaries I

Below, `${module}_python_file_count` placeholders expand per subdirectory at render-time.

`infrastructure.autoresearch` (11 files) |

Readiness planner, validation CLI, and report models for AutoResearch-style project promotion (`infrastructure/autoresearch/`).

infrastructure.benchmark (4 files) |

Template harness scoring and comparative gate helpers exercised in CI smoke paths.

infrastructure/config (non-package subdirectory) |

Repository-wide YAML templates and secure manifests
(.env.template, hardened defaults referenced by Docker + CLI).
config/ carries no __init__.py, so it is a configuration
subdirectory rather than an importable package.

infrastructure.core (133 files) |

Checkpointing, logging, pipeline YAML parsing, telemetry bridges, filesystem helpers, hardened exceptions. Everything else imports logging + error taxonomy from here first.

infrastructure.doctor (14 files) |

Checkout diagnose/fix/undo repairs for broken local workspace states.

infrastructure.docker (0 files) |

Pinned images / compose scaffolding for reproducible CI + remote builds.

infrastructure.documentation (23 files) |

Figure registries plus glossary tooling feeding manuscript automation.

infrastructure.fonds (6 files) |

Resource pool management for curated fonds (tracked reference datasets, bibliographic collections, and evidence corpora). Fonds mirror projects/templates/ with git-tracked templates/* exemplars and sidecar-linked private lifecycle folders.

infrastructure.llm (55 files) |

Ollama integrations, sanitization adapters, templated reviewer flows. **Literature ingestion now lives primarily in search/literature + citation helpers in reference/.**

infrastructure.methods (8 files) |

Deterministic methods-orchestration contracts (`MethodStage`, `MethodsOrchestrationPlan`, `MethodsIssue`): builds and validates an ordered methods plan for a research project so the manuscript's "Methods" track stays bound to executable stages.

`infrastructure.orchestration` (12 files) |

`python -m infrastructure.orchestration` exposes interactive menus, subprocess wiring for thin shell wrappers (`run.sh`, `secure_run.sh`), and stubs used in CI for menu parsing tests.

infrastructure.project (42 files) |

Canonical discovery (`discover_projects`) enforcing `src/` + `tests/`, slug validation, nested WIP namespaces.

infrastructure.prose (9 files) |

Readability metrics + Markdown tooling for prose-centric manuscripts / CI gates.

infrastructure.provenance (8 files) |

Content-addressed provenance DAG. Records artifact lineage (which run produced which file, from which inputs) as a verifiable graph of artifact/run/source/claim nodes connected by produced/consumed/derived-from/supports/refutes edges. Includes a structured Review system with severity (blocking/major/minor/info) and verdict (refutes/supports). Features a CLI and pipeline integration hooks for automatic lineage recording after every stage.

infrastructure.publishing (84 files) |

Metadata models, APA/BibTeX/MLA formatters, optional Zenodo clients.

infrastructure.reference (16 files) |

Citation/BibTeX parsing + conversion utilities leveraged by manuscripts and retrieval scripts.

infrastructure.rendering (81 files) |

Pandoc shim, Unicode/XeLaTeX postprocessors, combined
PDF/HTML/slide exporters.

`infrastructure.reporting` (58 files) |

Parses pytest + coverage artefacts for dashboards; pairs with Stage 01 summaries and downstream executive exports.

infrastructure.research (3 files) |

Seven-stage research workflow

(SCOPE→LITERATURE→REASON→DESIGN→COMPUTE→SYNTHE

defined as typed ResearchStage data classes with explicit outputs, skills, and template commands per stage. Includes a full PRISMA-adapted literature review prompt and a research workflow prompt referencing template/ infrastructure commands.

infrastructure.rules (6 files) |

Governance rules layer for validating project lifecycle transitions, sidecar sync policies, and pipeline gate orchestration. Rules mirror projects/templates/ with git-tracked templates/* exemplars.

infrastructure.scientific (4 files) |

Stability probing, benchmarking hooks—consumed heavily by optimization exemplars (template_code_project scripts).

Two-tier search architecture: the `literature/` client stack (`client.py`, `backends`, `caches`) powers literature search with arXiv, Crossref, local, and Paperclip backends. `connectors/` exposes the built-in scientific database adapters through a uniform `Connector Registry`; OpenAlex, UniProt, PDB, Semantic Scholar, European PMC, bioRxiv, and other registered adapters share `list/search` CLI commands with HTTP timeout, retry, and TTL caching. `monid/` provides an opt-in paid gateway (`discover / inspect / run`) with offline USD/1k pricing reference in `monid/PRICING.md`. `deep_research/` dispatches long-horizon provider-neutral manuscript research (PAID, opt-in). The live registry, not this prose, is authoritative for connector count.

infrastructure.sia (10 files) |

Generic Self-Improving-AI loop utilities: task-layout validation, execution harness, and metric capture reused by `template_sia` (fixture-replay by default).

infrastructure.skills (8 files) |

Discovers SKILL.md frontmatter → .cursor/skill_manifest.json.

infrastructure.steganography (13 files) |

Watermark overlays, hashing companions triggered by secure pipeline path.

infrastructure.tools (6 files) |

Invocable tool definitions registered by resource-pool governance;
tools mirror projects/templates/ with git-tracked
templates/* exemplars.

infrastructure.validation (84 files) |

Markdown + PDF + integrity CLIs underpinning Stage 04 diagnostics.

infrastructure/logrotate.d (0 files) |

Operational templates for deployments (documentation-first; intentionally minimal Python footprint).

Documentation maturity: Coverage statements in Results pull from introspection—not hand-maintained denominators—so newly promoted modules automatically flow into manuscripts after `generate_manuscript_metrics.py`.

FAIR+RSE linkage: MCP-ready SKILL.md artefacts align with evaluator heuristics (executability + metadata richness) emphasized by FAIRsoft guidance (Garijo et al. 2024).